

Matchmatic

AI-Powered Job Application Automation • Vietnam & United States

Victoria DevOps · Infra · Cloud	Heulwen Full-Stack · Frontend · AI Prompts	13 Weeks · Part-time Gemini 2.0 Flash · GCP · Supabase
---	--	--

SECTION 1 — PROJECT OVERVIEW

1.1 Project Purpose

Matchmatic was built from a real, painful workflow: manually browsing job boards, copy-pasting job descriptions into Gemini for ATS analysis, copying that output into ChatGPT to strip the AI robotic tone, then applying by hand — job by job. Every step is context-switching, repetition, and wasted time.

Matchmatic eliminates that entire manual loop. It is an end-to-end agentic pipeline that automatically finds relevant jobs posted in the last 24 hours, scores them against your resume with a detailed match analysis, flags US visa compatibility if you need it, tailors your resume through a three-stage AI pipeline, submits the application with a Playwright browser bot, and tracks every outcome in a kanban dashboard. The only thing you decide is whether to approve the tailored resume and confirm the submission. Everything else is automated.

This is purpose-built for two specific markets: Vietnam (VietnamWorks, TopCV) and the United States (Indeed). The visa detection layer is a first-class feature — not an afterthought — because applying to a role that requires US citizenship is a waste of time for international candidates on F1-OPT.

1.2 Project Aim

- Cut time per application from 30–45 minutes of manual work to under 2 minutes of human decision-making
- Apply to more jobs faster without sacrificing resume quality — every submission is tailored to that specific JD
- Give F1-OPT and international applicants a clear visa compatibility signal before wasting a full application cycle
- Store every application, resume version, and outcome in one place — no more spreadsheets and email threads
- Build a feedback loop where the system learns from your actual interview outcomes and improves match scoring over time

- Serve as a flagship portfolio project for both team members demonstrating AI Engineering, agentic orchestration, and cloud DevOps

1.3 MVP Scope

MVP Definition

The MVP is complete when a user can: (1) upload their resume once, (2) set preferences — country filter and visa toggle, (3) have the system automatically surface scored jobs, (4) approve a tailored resume, (5) auto-submit the application, and (6) see it tracked on the dashboard. All within one session. Zero manual copy-pasting.

Included in MVP

- Job scraper: Indeed US and VietnamWorks — jobs posted within 24 hours only
- ATS match scoring: percentage score (0–100) with HIGH / MEDIUM / LOW badge (green / yellow / red)
- Visa compatibility analysis for F1-OPT, H1B, and PR signals — optional toggle per user
- Detailed match report: matching skills, missing skills, experience alignment, and a substantive written analysis (not just a 3-line summary — see Section 5 for prompt specification)
- Three-agent resume tailoring pipeline: ATS Optimizer → Humanizer → QA Checker
- Resume diff viewer: original vs tailored, side by side with changes highlighted
- User approval step: no submission happens without explicit user confirmation
- Auto-apply via Playwright: Indeed Easy Apply only for MVP
- Application tracker: kanban board with Applied, Interview, Offer, Rejected columns
- Resume version storage: every tailored resume is saved and linked to the application it was used for

Out of Scope — Post-MVP

- LinkedIn scraping (anti-bot detection makes this a Phase 2 problem)
- Cover letter generation
- Interview preparation assistant
- Multi-user SaaS mode and billing
- Mobile app
- ML feedback loop retraining (data collection happens in MVP; reweighting is Phase 2)

SECTION 2 — TECH STACK

2.1 Full Stack

Layer	Technology	Purpose	Cost
Frontend	React 18 + Vite + TypeScript	Web UI, job feed, tracker dashboard, settings	Free
Styling	Tailwind CSS	Utility-first UI styling	Free
State management	Zustand	Lightweight global state	Free
HTTP client	Axios	API calls from React to FastAPI	Free
Kanban	@hello-pangea/dnd	Maintained fork of react-beautiful-dnd for tracker	Free
Diff viewer	react-diff-viewer	Show original vs tailored resume changes	Free
Backend	FastAPI (Python 3.11)	REST API + agent orchestration	Free
ASGI server	Uvicorn	Runs FastAPI in production	Free
Task queue	Celery + Redis	Async background job processing	Free (local Docker)
Database	PostgreSQL via Supabase	All persistent data	Free tier
File storage	Supabase Storage	Resume PDFs — original and tailored versions	Free tier (1 GB)
Cache	Redis (Upstash in prod)	Job dedup, Celery broker, analysis result cache	Free tier
Auth	Supabase Auth (Google OAuth)	User login — no password management needed	Free
Scraper	Playwright (Python)	Browser automation: scraping + auto-apply	Free
Scheduler (local)	APScheduler	Trigger scraper every 6 hours in dev	Free
Deployment	Google Cloud Run	Serverless containers — pay per request	~\$0 dev / \$5–15 prod
Scheduler (prod)	Google Cloud Scheduler	Trigger scraper in production on cron	Free tier (3 jobs)
Secrets	Google Secret Manager	Store API keys securely in production	Free tier
Monitoring	Google Cloud Logging	View logs and errors in production	Free tier
CI/CD	GitHub Actions	Auto-test and deploy on push to main	Free (2,000 min/month)
Containers	Docker + Docker Compose	Local dev environment — one command to run all	Free
Version control	GitHub (private repo)	Code + collaboration	Free

2.2 AI / LLM Stack

Agent	Model	Task	Est. cost per call
ATS Scorer	Gemini 2.0 Flash	Score resume vs JD, return structured JSON	~\$0.001
Visa Analyzer	Gemini 2.0 Flash	Detect OPT/H1B/PR signals in JD	~\$0.0005
ATS Optimizer	Gemini 2.0 Flash	Rewrite resume with JD keywords, keep all metrics	~\$0.003
Humanizer	Gemini 2.0 Flash	Strip AI jargon, natural confident voice	~\$0.003
QA Checker	Gemini 2.0 Flash	Validate output: 1-page, metrics intact, no jargon	~\$0.002
Report Writer	Gemini 2.0 Flash	Generate substantive written match analysis for user	~\$0.001

Total AI cost per application

All 6 agent calls together cost approximately \$0.01 USD per application.
Google AI Studio free tier = 1,500 requests/day free. During all 11 weeks of development you pay \$0.
At 50 applications/month in production: ~\$0.50 in AI costs.

2.3 Python Libraries

Library	Version	Purpose
fastapi	latest	Backend REST API framework
uvicorn	latest	ASGI server to run FastAPI
playwright	latest	Browser automation — scraping and auto-apply
google-generativeai	latest	Official Gemini API SDK
celery	latest	Async task queue
redis	latest	Cache and Celery message broker
sqlalchemy	latest	ORM for PostgreSQL
alembic	latest	Database migration tool
pydantic	v2	Data validation and JSON schema definitions
python-dotenv	latest	Load .env variables at runtime
supabase	latest	Supabase client for DB and Storage
apscheduler	latest	Local job scheduler (scraper trigger)
python-docx	latest	Parse DOCX resume uploads
pypdf2	latest	Extract text from PDF resume uploads
weasyprint	latest	Generate final resume PDFs from HTML
tenacity	latest	Retry logic for Gemini API calls
loguru	latest	Structured logging throughout the backend

SECTION 3 — USER FLOW

Complete User Journey

The following describes every step a user takes from first login through to their application being tracked on the dashboard.

Step 1 — Onboarding

- User signs in with Google via Supabase Auth — no password management required
- User uploads their base resume as a PDF or DOCX file
- User opens the Settings page and configures their session:
 - Target market: Vietnam only / US only / Both
 - Visa check toggle: ON (activates visa analyzer for every US job) or OFF
 - Job title keywords they are targeting: e.g. "DevOps Engineer, Cloud Engineer, SRE"
 - Minimum match score to surface in the feed: e.g. 70% — jobs below this are filtered silently

These settings are saved to the users table and passed as context to every AI call that follows. The visa toggle is the most important setting — it controls whether the Visa Analyzer agent runs at all.

Step 2 — Automated Job Discovery (background, every 6 hours)

- Scraper visits Indeed US and VietnamWorks automatically on a schedule
- For each job it collects: title, company, location, full JD text, date posted, and source URL
- Deduplication check: if this URL is already in the database, skip it
- Freshness filter: if the job was posted more than 24 hours ago, skip it
- New jobs are saved to the jobs table with status = "pending" and added to the Celery processing queue

This step runs entirely in the background. The user does not trigger it and does not wait for it. By the time a user opens the app, there will already be fresh jobs with scores ready to view.

Step 3 — AI Analysis (triggered automatically per new job)

- Celery worker picks up each pending job from the queue
- ATS Scorer: sends the job description and the user's stored resume text to Gemini, receives a structured JSON response with score, match level, skills breakdown, and a detailed written analysis
- Visa Analyzer: runs only if the user's visa toggle is ON — reads the JD for sponsorship signals and returns OPEN, CITIZEN_ONLY, or UNCLEAR with the specific phrase from the JD that triggered the verdict
- Report Writer: generates a substantive written analysis of the match — not a 3-line summary, but a real paragraph explaining why this job is or is not a good fit, what the candidate's strongest selling points are for this role, what gaps exist, and whether applying is worth the effort
- All results are saved to the job_analyses table linked to both the job and the user
- Jobs scoring below the user's minimum threshold are silently excluded from the feed

Step 4 — User Review (Job Feed)

- User opens the Jobs page — cards sorted by match score descending, newest first
- Each card shows: job title, company name, match score badge (green HIGH / yellow MEDIUM / red LOW), visa signal if toggle is on, time since posted
- User clicks a card to see the full match report: score breakdown, matching skills (green chips), missing skills (red chips), visa verdict with evidence quote, and the full written analysis
- User clicks "Tailor Resume" to proceed with that job, or "Skip" to dismiss it from their feed

Step 5 — Resume Tailoring (three-agent pipeline)

- Agent 1 — ATS Optimizer: rewrites the resume to closely match the JD, integrating keywords naturally and preserving every number and metric from the original
- Agent 2 — Humanizer: takes Agent 1's output and rewrites it to sound like a real experienced professional wrote it — no jargon, no AI tells, varied sentence lengths
- Agent 3 — QA Checker: validates the output — checks word count, verifies all original metrics survived, scans for banned words, and returns either PASS or FAIL with a corrected version if FAIL
- If QA fails twice, the user is shown the best available version with the issues listed for manual review
- User sees the resume diff view: original on left, tailored on right, additions in green, removals in red strikethrough
- User can edit inline before approving — the word count indicator updates in real time

Step 6 — Apply

- User clicks "Confirm and Apply" — this is the only action required
- Playwright browser bot opens the job URL, fills in the required form fields, uploads the tailored resume PDF, and submits
- If a CAPTCHA appears, the bot pauses and notifies the user to complete it manually
- Application event logged to the applications table with a timestamp
- Job card moves to the "Applied" column on the tracker automatically

Step 7 — Track

- Tracker dashboard shows all applications in four columns: Applied → Interview Scheduled → Decision Pending → Closed
- User drags cards between columns manually as their status changes
- Each card shows: company, job title, applied date, match score badge, days elapsed
- Clicking a card opens a detail panel: original JD, the exact tailored resume that was submitted, notes field, and outcome buttons
- Outcome buttons: "Got Offer", "Rejected", "Ghosted" — recording these feeds the future feedback loop
- Stats bar at top: total applied, total interviews, response rate percentage

SECTION 4 — DATABASE SCHEMA (ERD)

PostgreSQL hosted on Supabase free tier. All tables use UUID primary keys. Supabase Auth handles user identity — the users table extends the auth.users table with app-specific settings.

Table: users

Column	Type	Description
id	UUID PK (from Supabase Auth)	Links to Supabase Auth user
email	TEXT NOT NULL UNIQUE	User's email address
full_name	TEXT	Display name
target_market	TEXT	"vietnam" "us" "both"
visa_check_enabled	BOOLEAN DEFAULT false	Whether to run Visa Analyzer on US jobs
min_match_score	INTEGER DEFAULT 70	Minimum ATS score to surface jobs in feed
job_keywords	TEXT[]	Array of job title keywords to search for
resume_url	TEXT	Supabase Storage path to base resume PDF
resume_text	TEXT	Extracted plain text of base resume (used for AI calls)
created_at	TIMESTAMPTZ DEFAULT now()	Account creation timestamp
updated_at	TIMESTAMPTZ	Last settings update

Table: jobs

Column	Type	Description
id	UUID PK	Unique job identifier
source	TEXT	"indeed_us" "vietnamworks" "topcv"
external_id	TEXT	Original job ID from the source platform
url	TEXT NOT NULL UNIQUE	Source job posting URL — used for deduplication
title	TEXT NOT NULL	Job title
company	TEXT NOT NULL	Hiring company name
location	TEXT	City and country
country	TEXT	"us" "vietnam"
description_raw	TEXT	Full job description text — sent to AI agents
posted_at	TIMESTAMPTZ	When the job was posted on the source site
scraped_at	TIMESTAMPTZ DEFAULT now()	When Matchmatic scraped it
is_processed	BOOLEAN DEFAULT false	True once the AI analysis has been queued

Table: job_analyses

Column	Type	Description
id	UUID PK	Unique analysis record
job_id	UUID FK → jobs.id	Which job was analyzed
user_id	UUID FK → users.id	Whose resume was used for this analysis
ats_score	INTEGER	Match percentage 0–100
match_level	TEXT	"HIGH" "MEDIUM" "LOW"
matching_skills	TEXT[]	Skills found in both JD and resume
missing_skills	TEXT[]	Skills required by JD but absent from resume
experience_match	TEXT	"strong" "partial" "weak" — years and seniority alignment
visa_compatible	BOOLEAN NULLABLE	Null if visa check disabled, true/false/null otherwise
visa_signal	TEXT	"open" "citizen_only" "unclear" null
visa_evidence	TEXT	Exact phrase from JD that triggered the visa verdict
summary	TEXT	Substantive written match analysis — multiple paragraphs, not just 3 sentences
analyzed_at	TIMESTAMPTZ DEFAULT now()	When the analysis was completed

Table: applications

Column	Type	Description
id	UUID PK	Unique application record
user_id	UUID FK → users.id	Who applied
job_id	UUID FK → jobs.id	Which job was applied to
status	TEXT	"applied" "interview" "pending" "closed"
tailored_resume_url	TEXT	Supabase Storage path to the tailored PDF that was submitted
tailored_resume_text	TEXT	Plain text of tailored resume — for diff view
ats_score_before	INTEGER	Original match score before tailoring
ats_score_after	INTEGER	Score after tailoring — QA checker validates this improved
applied_at	TIMESTAMPTZ	Exact timestamp of submission
notes	TEXT	User-written notes — interview prep, follow-up reminders
outcome	TEXT NULLABLE	"offer" "rejected" "ghosted" null
updated_at	TIMESTAMPTZ	Last status change

Table: resume_versions

Column	Type	Description
id	UUID PK	Unique version record
application_id	UUID FK → applications.id	Which application this version belongs to
user_id	UUID FK → users.id	Who owns this resume
version_type	TEXT	"original" "ats_optimized" "humanized" "final"
content_text	TEXT	Plain text of this version — used for diff and debugging
storage_url	TEXT	Supabase Storage URL for the PDF of this version
created_at	TIMESTAMPTZ DEFAULT now()	When this version was generated

Table: scraper_runs

Column	Type	Description
id	UUID PK	Unique run record
source	TEXT	Which site was scraped
jobs_found	INTEGER	Total jobs seen on that run
jobs_new	INTEGER	Jobs that passed dedup and freshness filter
status	TEXT	"success" "failed" "partial"
error_message	TEXT NULLABLE	Error details if status is failed
started_at	TIMESTAMPTZ	When the run began
finished_at	TIMESTAMPTZ	When the run completed

Key Relationships

- users → job_analyses: one user has many analyses (one per job they are matched against)
- users → applications: one user has many applications
- jobs → job_analyses: one job can be analyzed for multiple users independently
- applications → resume_versions: one application has up to 4 versions (original, ats_optimized, humanized, final)

SECTION 5 — AI AGENTS AND PROMPTS

These are the six AI agents that power Matchmatic. Each agent is a single Gemini 2.0 Flash call with a specific prompt. All agents return JSON except the Humanizer, which returns plain text. Test every prompt with at least 10 real job descriptions before moving on to the next phase.

Critical note on the match summary

The original blueprint specified a "3-sentence summary". This has been upgraded.

The Report Writer agent must produce a substantive analysis — typically 3–4 solid paragraphs covering: (1) overall fit assessment and headline verdict, (2) the candidate's strongest selling points for this specific role, (3) skill and experience gaps and how significant they are, and (4) a clear recommendation on whether applying is worth the time.

This is the feature that makes Matchmatic genuinely useful versus other tools. Make it count.

Agent 1 — ATS Scorer

Purpose: Score how well the resume matches the JD and return structured data the UI can display directly.

You are an ATS (Applicant Tracking System) expert and senior technical recruiter. Given the job description and resume below, analyze how well the candidate matches and return ONLY valid JSON with this exact structure: { "score": <integer 0-100>, "match_level": <"HIGH" | "MEDIUM" | "LOW">, "matching_skills": [<array of strings>], "missing_skills": [<array of strings>], "experience_match": <"strong" | "partial" | "weak">, "improvement_tip": "<one specific, actionable suggestion>" } Scoring: 80-100 = HIGH (strong candidate, likely to pass ATS). 60-79 = MEDIUM (decent match, worth applying with tailoring). Below 60 = LOW (significant gaps). Count: exact keyword overlap, required years of experience, seniority alignment, and technical stack match. Return nothing except the JSON object.

Agent 2 — Report Writer

Purpose: Generate a substantive written analysis of the match — the insight the user actually needs to make a decision.

You are a senior career advisor and recruiter. Given the job description, the candidate's resume, and the ATS analysis JSON below, write a thorough match analysis. Structure your response as 3-4 paragraphs: Paragraph 1 – Headline verdict: Is this a strong match? State the score context and whether applying is worth the candidate's time. Paragraph 2 – Strongest selling points: What specifically in this candidate's background makes them compelling for this role? Be specific – name the experiences, projects, or skills. Paragraph 3 – Gaps and how serious they are: What is the candidate missing? Are these deal-breakers or easily explained? How significant is each gap in the context of this role? Paragraph 4 – Recommendation: Should they apply? If yes, what should they emphasize in the tailored resume and any cover communication? If no, why not? Write in plain English. No bullet points. No jargon. Sound like a thoughtful advisor, not a robot. This analysis will be read by the candidate to help them decide whether to spend time applying.

Agent 3 — Visa Analyzer (only runs if toggle is ON)

Purpose: Detect whether a US job is accessible to F1-OPT international students.

You are analyzing a US job description for work authorization signals. Return ONLY valid JSON: { "visa_signal": <"open" | "citizen_only" | "unclear">, "evidence": "<direct quote from the JD that led to this conclusion, or 'No explicit statement found' if none>", "fl_opt_compatible": <true | false | null>, "confidence": <"HIGH" | "MEDIUM" | "LOW"> } Signal meanings: "open" = job explicitly welcomes visa holders, mentions sponsorship, or says all work authorizations welcome. "citizen_only" = job requires US citizenship, permanent residency, active security clearance, or says no sponsorship. "unclear" = no mention of work authorization at all. Return nothing except the JSON object.

Agent 4 — ATS Optimizer

Purpose: Rewrite the resume to match the JD keywords while preserving every original metric.

You are a professional resume writer specializing in ATS optimization. Rewrite the resume below to closely match the job description. Rules: (1) Keep every metric and number from the original exactly as written – never replace a specific number with a vague word like "significantly". (2) Naturally integrate keywords from the job description – do not keyword-stuff or add skills the candidate does not have. (3) Keep the same structure and sections as the original resume. (4) Target one page – aim for under 650 words total. (5) Do not invent experience or claims that are not in the original. Return the full rewritten resume text only. No commentary, no explanation.

Agent 5 — Humanizer

Purpose: Make the ATS-optimized resume sound like a real, experienced professional wrote it.

You are an elite executive resume writer. Rewrite the resume draft below so it sounds deeply human, confident, and authoritative – not AI-generated. Rules: (1) Vary sentence length throughout – mix short punchy statements with longer descriptive ones. A page of uniform sentence lengths is the biggest tell. (2) Remove ALL of these words regardless of context: spearheaded, leveraged, synergy, synergized, passionate, dynamic, innovative, results-driven, detail-oriented, proactive, strategic thinker, thought leader, utilized, facilitated. (3) Keep every metric and number exactly as written. Never soften a statistic. (4) Use strong active verbs but make them feel natural and specific to the context, not generic action verbs bolted on. (5) Focus on business impact and ownership – not just responsibilities. (6) Stay under 650 words. Return the rewritten resume text only.

Agent 6 — QA Checker

Purpose: Validate the final resume and auto-fix any failures before showing to the user.

You are a resume quality checker. Given the original resume, the tailored resume, and the job description, return ONLY valid JSON: { "word_count": <integer>, "fits_one_page": <true | false – true if under 650 words>, "all_metrics_preserved": <true | false>, "missing_metrics": [<list any numbers from original not found in tailored>], "banned_words_found": [<list any of these if present: spearheaded, leveraged, synergy, passionate, dynamic, innovative, utilized, facilitated>], "ats_improvement_clear": <true | false>, "verdict": <"PASS" | "FAIL">, "fixed_resume": "<if FAIL, return the fully corrected resume text here. If PASS, return empty string>" } If verdict is FAIL, fix all identified issues in fixed_resume. Maximum 2 auto-fix attempts – after 2 failures, return FAIL and let the user decide. Return nothing except the JSON object.

SECTION 6 — SYSTEM ARCHITECTURE

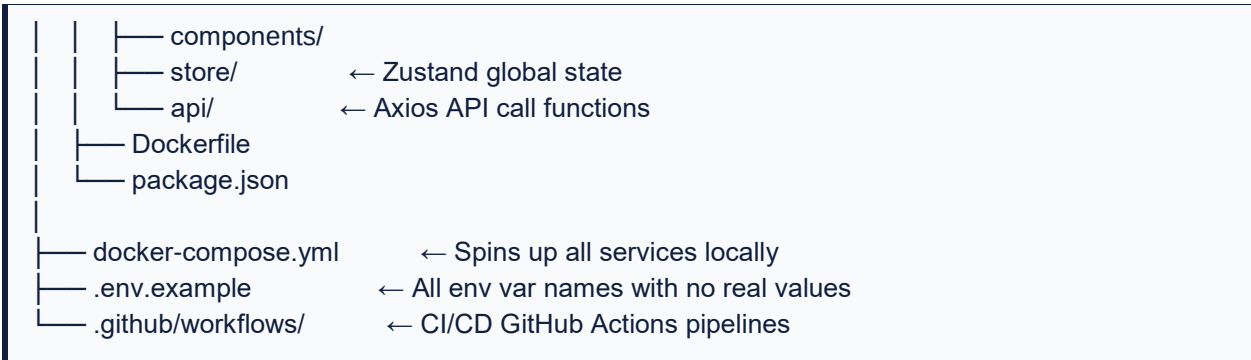
6.1 Repository Structure

Monorepo: matchmatic/

```

matchmatic/
├── backend/                ← Python FastAPI application
│   ├── app/
│   │   ├── main.py        ← FastAPI entry point
│   │   ├── config.py      ← Environment variable config
│   │   ├── database.py    ← SQLAlchemy engine + session factory
│   │   ├── models/       ← SQLAlchemy models (users, jobs, applications...)
│   │   ├── routers/      ← API route handlers
│   │   │   ├── jobs.py
│   │   │   ├── applications.py
│   │   │   ├── resume.py
│   │   │   └── users.py
│   │   ├── agents/       ← All six AI agent logic files
│   │   │   ├── gemini_client.py ← Shared Gemini wrapper with retry
│   │   │   ├── ats_scorer.py
│   │   │   ├── report_writer.py
│   │   │   ├── visa_analyzer.py
│   │   │   ├── ats_optimizer.py
│   │   │   ├── humanizer.py
│   │   │   └── qa_checker.py
│   │   ├── prompts/      ← All prompt strings as Python constants
│   │   ├── scraper/      ← Playwright scrapers
│   │   │   ├── base_scraper.py
│   │   │   ├── indeed.py
│   │   │   └── vietnamworks.py
│   │   ├── tasks/        ← Celery task definitions
│   │   │   ├── scraper_task.py
│   │   │   ├── analysis_task.py
│   │   │   └── apply_task.py
│   │   └── utils/        ← PDF parse, resume diff, text extraction
│   ├── alembic/          ← Database migrations
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/              ← React + Vite + TypeScript
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Jobs.tsx   ← Job feed with match cards
│   │   │   ├── Tracker.tsx ← Kanban application tracker
│   │   │   ├── Settings.tsx ← User preferences page
│   │   │   └── ResumeReview.tsx ← Diff view before apply

```



6.2 Environment Variables (.env)

Variable	Value in dev	Description
GEMINI_API_KEY	From Google AI Studio	Gemini API access key
SUPABASE_URL	From Supabase project settings	Supabase project URL
SUPABASE_ANON_KEY	From Supabase project settings	Public anon key for client-side calls
SUPABASE_SERVICE_ROLE_KEY	From Supabase project settings	Server-side only — full DB access
DATABASE_URL	PostgreSQL connection string	Used by SQLAlchemy
REDIS_URL	redis://localhost:6379	Local Docker Redis; Upstash URL in prod
SECRET_KEY	Random 32-character string	Session signing key
ENVIRONMENT	"development" or "production"	Controls logging level and error detail
FRONTEND_URL	http://localhost:5173	CORS allow-list — Vercel URL in prod

SECTION 7 — 13-WEEK ROADMAP

How to Read This Roadmap

Each week table has three columns. Victoria's tasks are in blue. Heulwen's tasks are in green. Tasks labeled BOTH are done together — usually a sync call, review session, or end-to-end test. Start every week with a 30-minute sync. End every Friday with a brief demo of what you each built, even if it is rough.

Week	Victoria — DevOps / Infra / Cloud	Heulwen — Full-Stack / Frontend / AI
------	-----------------------------------	--------------------------------------

Before Week 1 — Together (one 2-hour session)

- Create GitHub repo: matchmatic (private, monorepo). Invite each other as collaborators.
- Set up branch strategy: main (protected, requires PR) and dev (working branch).
- Create /backend and /frontend folder placeholders with README stubs.
- Create .env.example with all variable names from Section 6.2 (no real values).
- Sign up for Supabase — create project, copy the connection string.
- Both go to aistudio.google.com — each get a Gemini API key.
- Install on both machines: Docker Desktop, Python 3.11+, Node 20+, VS Code.
- Run docker-compose up with a basic PostgreSQL + Redis compose file — confirm both services respond.

Phase 1 — Foundations (Weeks 1–2)

Week 1 Environment	- Write docker-compose.yml with PostgreSQL, Redis, backend, and Celery worker services - Set up backend FastAPI project using the folder structure from Section 6.1 - Write database.py with SQLAlchemy engine + session factory connected to Supabase - Write all 5 SQLAlchemy models matching the ERD (users, jobs, job_analyses, applications, resume_versions, scraper_runs) - Run first Alembic migration: alembic init + alembic revision --autogenerate + alembic upgrade head - Set up GitHub Actions CI: lint with ruff + run pytest on every push to dev - Configure .env template and verify FastAPI health check endpoint responds at /health	- Set up Vite + React + TypeScript project in /frontend - Install and configure Tailwind CSS, Zustand, React Router v6, Axios - Create page stubs: Jobs.tsx, Tracker.tsx, Settings.tsx, ResumeReview.tsx - Build navbar component and routing in App.tsx — navigation between pages should work - Set up Supabase Auth with Google OAuth — user can sign in and sign out - Build the Settings page UI: country selector, visa toggle, keyword input, min score slider - Wire Settings form to PATCH /users/me/settings endpoint (Victoria builds the endpoint)
--------------------------------------	---	--

Week 2

Scraper + API

- Install Playwright: `pip install playwright` & `playwright install chromium`
- Write `base_scraper.py` — abstract class: `launch_browser()`, `close_browser()`, `save_job()`, `check_duplicate()`
- Write `vietnamworks.py` — navigate search results, extract: title, company, JD text, URL, posted_at
- Write `indeed.py` — navigate Indeed US search results, extract same fields
- Add 24-hour freshness filter: skip any job where posted_at is more than 24 hours ago
- Add Redis deduplication: query Redis for URL before saving — skip if exists
- Wrap scraper in Celery task (`tasks/scraper_task.py`) and set APScheduler to trigger every 6 hours
- Write `scraper_runs` logging: record start time, jobs_found, jobs_new, status, any errors
- Build GET `/jobs` endpoint: paginated, filterable by country and match_level
- Build GET `/jobs/{id}` endpoint: returns full job detail
- Build POST `/resume/upload` endpoint: accepts PDF, extracts text, stores in Supabase Storage
- Build the Jobs feed page in React: fetches from GET `/jobs`, renders job card list
- Build the job card component: shows title, company, location, posted time
- Build the resume upload component: drag-and-drop or file picker, sends to POST `/resume/upload`
- Add basic country filter to the job list UI

Phase 2 — AI Analysis Engine (Weeks 3–5)

Week 3

Gemini Client

- Write `gemini_client.py`: class `GeminiClient` with `call_model(prompt, system_prompt)` method
- Add retry logic using tenacity: 3 retries with exponential backoff on API failure
- Add token usage logging: log input_tokens and output_tokens per call using loguru
- Write `analysis_task.py`: Celery task `analyze_job(job_id, user_id)` — runs ATS Scorer + Visa Analyzer, saves to `job_analyses`
- Wire scraper task to auto-trigger `analyze_job` for each new job found
- Add rate limit guard: track calls per minute, queue if approaching Gemini free tier limit
- Write and test the ATS Scorer prompt — it must return valid parseable JSON every single time
- Write and test the Visa Analyzer prompt — confirm it returns correct signals on 10 real US JDs
- Build the job card component update: add match score badge (green HIGH / yellow MEDIUM / red LOW)
- Build the visa badge: OPEN (green), CITIZEN ONLY (red), UNCLEAR (yellow), NOT CHECKED (gray)
- Build skeleton loading states on job cards so UI does not feel broken while analysis runs
- Add GET `/jobs/feed` endpoint filter: only return jobs at or above user's min_match_score

Week 4

Match Report

- Write `report_writer.py` agent using the Agent 2 prompt from Section 5
- Wire Report Writer to run after ATS Scorer — saves result to summary column in `job_analyses`
- Build the match report modal: opens when user clicks a job card
- Modal shows: score percentage, color badge, matching skills (green chips), missing skills (red chips)

	• Write GET /jobs/{id}/report endpoint: returns full job_analyses record including the written analysis • Add Redis caching for analysis results: cache for 1 hour to avoid re-analyzing same job • Add admin endpoint GET /admin/scrapper-runs to see scraper run history (for debugging) • Write unit tests for gemini_client.py: mock the API, assert retry logic works correctly	• Modal shows: experience match rating, visa status with evidence quote if toggle is on • Modal shows: the full written analysis from Report Writer — render as formatted paragraphs, not a blob of text • Add "Tailor Resume" (primary) and "Skip" (secondary) buttons at the bottom of the modal • Skip marks job as dismissed for that user — add dismissed boolean to job_analyses
Week 5 AI Quality Review	• Run 20 real job descriptions through both ATS Scorer and Report Writer — review JSON quality • Tune retry logic: if Gemini returns malformed JSON after 3 retries, mark analysis as failed and surface error to frontend • Load test Celery queue: simulate 50 jobs arriving at once, confirm worker handles it without crashing • Add database indexes on: user_id, job_id, status, country, ats_score — queries will slow down without these • Security pass: all API endpoints require valid JWT — verify with curl tests	• Review all 20 match reports with Victoria — grade quality and note where prompts need improvement • Refine ATS Scorer and Report Writer prompts based on review — this day is non-negotiable • Build the job feed sorting: HIGH match first, then MEDIUM, then LOW, then by newest within each group • Add search bar: filter by keyword or company name • Add "Mark as Not Interested" button to dismiss from feed permanently • End-to-end test: scraper finds a job → AI scores it → appears in feed with correct badge and written analysis
BOTH	• Full end-of-phase review: does the job feed feel useful? Do the match reports give real insight? If no, fix prompts before moving on. • Check token usage logs: confirm total daily usage is well under 1,500 free tier limit.	

Phase 3 — Resume Pipeline (Weeks 6–8)

Week 6 Agent Chain	• Add resume_versions table migration using Alembic • Build resume_pipeline.py orchestrator: runs Agent 4 → Agent 5 → Agent 6 in sequence • After each agent runs, save intermediate output to resume_versions table (version_type: "ats_optimized", "humanized", "final") • If QA Checker returns FAIL with fixed_resume field populated, use that as input for one retry pass	• Write and test the ATS Optimizer prompt (Agent 4) — verify keywords are added naturally • Write and test the Humanizer prompt (Agent 5) — run on 5 samples, confirm banned words are gone • Write and test the QA Checker prompt (Agent 6) — verify JSON parses correctly and FAIL cases return fixed_resume • Write unit tests for each agent: mock Gemini response, assert JSON parsed and stored correctly
------------------------------	--	--

	- After 2 FAIL results, return the best available version and flag the specific issues for Heulwen's UI - Add pipeline status endpoint: GET /resume/pipeline-status/{job_id} — returns current step and whether complete	- Build the "Tailor Resume" button flow: clicking triggers pipeline, shows step-by-step progress indicator - Write POST /resume/tailor endpoint: accepts job_id, triggers pipeline as Celery task, returns task_id
Week 7 Diff + PDF	- Write utils/resume_diff.py: takes two text strings, returns list of added/removed/unchanged segments as JSON - Build PDF generation: use WeasyPrint to convert final resume text to a clean one-page PDF - Create default resume HTML template for WeasyPrint — test that output is actually one page - Store generated PDFs in Supabase Storage — save URL to resume_versions and applications tables - Write GET /resume/diff/{application_id} endpoint: returns diff JSON for the frontend to render	- Build ResumeReview.tsx page: original resume on left, tailored on right, full-width layout - Render diff: green background for added text, red strikethrough for removed text — use react-diff-viewer - Add word count indicator: live count as user edits, shows green when under 650 words - Add inline edit mode: user can click on the tailored side and edit text directly before approving - Add QA Checker results panel below diff: shows PASS/FAIL verdict, and lists any issues if FAIL - Add "Approve and Apply" button — only enabled if QA verdict is PASS or user has manually edited
Week 8 Pipeline Polish	- Comprehensive error handling in all agents: log every failure with the prompt that caused it - Add banned words config: a JSON file listing all banned words — easy to update without touching code - Add token cost logging per full pipeline run: sum input + output tokens across all 3 agents and log total - Performance test: full pipeline should complete in under 45 seconds — identify and fix bottlenecks	- Have 2-3 friends use the full resume tailoring flow — collect feedback on the diff view and quality of output - Fix any UX issues from user testing: confusing labels, missing feedback, broken states - Add "Resume History" page: user can see all tailored resume versions ever generated - Add toast notifications: "Resume tailored successfully", "Analysis failed — click to retry" - Add error states on all cards: "Analysis failed" with a retry button
BOTH	Prompt quality review: test full pipeline with 10 real job descriptions. Grade each output. Refine prompts until the quality is consistently good. Do not skip this.	

Phase 4 — Auto-Apply and Tracker (Weeks 9–11)

Week 9 Auto-Apply Bot	Write apply_task.py: Celery task that triggers Playwright bot — only runs after user confirms	Write auto_apply/indeed.py: Playwright script targeting Indeed Easy Apply
---------------------------------	---	---

	• Write application event logger: on successful Playwright submit, create row in applications table • Write POST /applications/submit endpoint: accepts job_id + resume_id, queues the Celery apply task • Build error handling: if Playwright submit fails, log screenshot path, set application status to "failed", alert frontend • Add "Applied" badge to job cards in the feed so user can see which jobs they already submitted	• Fill in fields: name (from user profile), email, phone, resume PDF upload • Add screenshot capture at each step — save to Supabase Storage for debugging • Add CAPTCHA detection: if CAPTCHA appears, pause bot and send notification to user to complete manually • Build the "Confirm and Apply" modal: shows company, job title, which resume version will be submitted • Build basic tracker page: simple list of all applications with status column
Week 10 Kanban Tracker	• Write GET /applications endpoint: returns all applications for logged-in user with job details joined • Write PATCH /applications/{id}/status: updates kanban column • Write PATCH /applications/{id}/notes: saves user notes • Write PATCH /applications/{id}/outcome: records offer / rejected / ghosted • Add VietnamWorks auto-apply support — different form layout than Indeed, test on 3 postings • Run 5 real auto-apply tests on Indeed test accounts — document any edge cases or failures	• Build Tracker.tsx with 4 columns: Applied, Interview Scheduled, Decision Pending, Closed • Implement drag-and-drop using @hello-pangea/dnd — on drop, call PATCH /applications/{id}/status • Each card shows: company, job title, applied date, match score badge, days since applied • Clicking a card opens a detail panel: JD text, tailored resume used, notes text area, outcome buttons • Outcome buttons: "Got Offer", "Rejected", "Ghosted" — call PATCH /applications/{id}/outcome • Add stats bar at top of tracker: total applied, total interviews, response rate percentage
Week 11 Polish + Security	• Full end-to-end integration test: scrape a real job → analyze → tailor → apply → confirm in tracker • Add health check monitoring: alert (log + frontend banner) if scraper has not run successfully in 8 hours • Performance test: job feed should load in under 2 seconds with 100+ jobs in database • Database index audit: verify indexes exist on all frequently queried columns • Security audit: rotate all test API keys, verify no secrets in git history, check all endpoints require auth	• Final UI polish pass: consistent spacing, typography, and color usage throughout the entire app • Add empty states for every page: no jobs found, no applications yet, resume not uploaded • Add error boundary components: if an API call fails, show a human-readable message not a white screen • Make the app responsive for laptop and tablet screen sizes • Write basic Playwright end-to-end UI test for the critical path: login → view job → tailor → review

Phase 5 — Deploy and Document (Weeks 12–13)

Week 12 Production Deploy	• Write Dockerfile for /backend — multi-stage: python:3.11-slim base, install dependencies, copy app • Push backend image to Google Artifact Registry • Deploy backend to Google Cloud Run — set all env vars via Secret Manager • Set up Google Cloud Scheduler to call the scraper endpoint every 6 hours in production • Set up GitHub Actions to build and deploy to Cloud Run on push to main • Configure Supabase for production: enable row-level security on all tables • Run full end-to-end test on the production URL — every step of the user flow • Configure CORS in FastAPI: allow the deployed Vercel frontend URL	• Build Vite production build — verify no TypeScript errors, no console warnings • Deploy frontend to Vercel: connect GitHub repo, set VITE_API_URL env var to Cloud Run URL • Add a landing page: one-paragraph description of Matchmatic, sign-in button • Fix any issues that appear only in production: CORS errors, missing env vars, Supabase RLS blocks • Run the full user journey on the live production URL and confirm it works end to end
Week 13 Docs + Launch	• Write deployment README: how to clone, configure env vars, run locally with Docker Compose • Document the CI/CD pipeline — what triggers a deploy, how to roll back • Do final security audit: rotate API keys, confirm no secrets in git history (use <code>git log --all -p grep -i key</code>) • Tag v1.0.0 release on GitHub • Set up Google Cloud Error Reporting for production error alerting	• Write user-facing README with screenshots, feature descriptions, and a demo GIF • Write project write-up for portfolio: problem, solution, tech choices, architecture decisions, what you would do differently • Record a 2-3 minute demo video of the full flow — use Loom or OBS • Create a demo GIF from the recording using ezgif.com — embed in README • Add MIT license file to repository
BOTH	• Write LinkedIn post announcing Matchmatic — tag the tech stack, include the demo GIF, link to GitHub. • Update both resumes with the project entry — see Section 8 for the exact wording to use.	

SECTION 8 — COST BREAKDOWN

Development Phase (Weeks 1–11) — Target: \$0

Service	Free Tier Limit	Your Usage in Dev	Cost
Google AI Studio	1,500 req/day	~50 req/day during dev	\$0
Supabase DB	500 MB storage, 2 GB BW	Well under in dev	\$0
Supabase Storage	1 GB	Under in dev	\$0
Redis (Docker)	Unlimited local	Local only	\$0
GitHub	Unlimited private repos	Full use	\$0
Docker Desktop	Free personal	Full use	\$0
GCP free credits	\$300 new account credit	Use for staging and testing	\$0

Production Phase (per month, personal use)

Service	Usage Estimate	Rate	Monthly Cost
Gemini 2.0 Flash	~500 applications/month	~\$0.01/app (6 agents)	~\$5
Google Cloud Run	~50 compute hours/month	Pay per request	~\$5–10
Supabase	Free tier until 500 MB DB	\$0 or \$25/month Pro	\$0 or \$25
Upstash Redis	Free tier: 10,000 req/day	\$0 or \$0.20/100k req	\$0–2
Cloud Scheduler	3 scheduled jobs (scraper)	Free tier	\$0
Vercel (frontend)	Static hosting	Free tier	\$0
Domain (optional)	matchmatic.io or similar	~\$12/year	~\$1
TOTAL			~\$11–43

SECTION 9 — HOW TO PRESENT THIS PROJECT

9.1 Victoria's Resume Entry (DevOps/Infra focus)

Matchmatic | AI-Powered Job Application Automation Platform | github.com/your-username/matchmatic

Architected and deployed a six-agent LLM pipeline on Google Cloud Run using Docker, Celery, and Redis — processing job analyses asynchronously against a PostgreSQL schema on Supabase.

Built Playwright-based web scrapers targeting Vietnam and US job markets with Redis deduplication, 24-hour freshness filtering, and APScheduler-based scheduling.

Engineered a CI/CD pipeline via GitHub Actions: automated testing on every push to dev, zero-downtime deployment to Cloud Run on merge to main.

Designed a PostgreSQL schema supporting multi-version resume tracking, outcome feedback loops, visa-aware job matching, and scraper run audit logging.

9.2 Heulwen's Resume Entry (Full-Stack/AI focus)

Matchmatic | AI-Powered Job Application Automation Platform | github.com/your-username/matchmatic

Engineered a three-stage AI resume tailoring pipeline (ATS Optimizer → Humanizer → QA Checker) using Gemini 2.0 Flash, with programmatic output validation and automatic retry on failure.

Built a Playwright auto-apply bot targeting Indeed Easy Apply, with CAPTCHA detection, screenshot capture for debugging, and a mandatory user confirmation step before submission.

Designed and implemented a React + TypeScript job feed with real-time ATS match scoring, visa compatibility badges, and a substantive AI-generated match analysis per job.

Built a full application tracker with drag-and-drop kanban board, outcome recording, and a resume diff viewer showing original vs tailored version side by side.

9.3 What to Say in Interviews

Victoria — sample interview talking point

"Matchmatic is a two-person project I built with a full-stack engineer. I owned the entire infrastructure layer — containerization with Docker, async job processing with Celery and Redis, CI/CD with GitHub Actions, and deployment to Google Cloud Run. The interesting infra problem was designing the Celery task chain so that each AI agent passes its output to the next without blocking the API, and the whole thing fails gracefully if any step in the chain breaks. I also built the scraper infrastructure with proper dedup and freshness filtering so we never re-process a job we have already seen."

Heulwen — sample interview talking point

"Matchmatic is a two-person project I built with a DevOps engineer. I owned the AI pipeline and the entire frontend. The most interesting engineering challenge was prompt design — getting Gemini to consistently return valid JSON across six different agents, each with different validation requirements. The QA Checker agent was particularly tricky because it has to detect whether the previous agent dropped a number from the original resume, which requires comparing strings across two texts and identifying missing metrics programmatically via the AI itself. The frontend challenge was building a

resume diff viewer that feels intuitive — showing what changed between the original and the tailored version without overwhelming the user."

9.4 What This Project Is — and Is Not

For both of you to say confidently

"This is AI Engineering — specifically Agentic AI Development. We are not training models. We are orchestrating pre-trained LLMs through well-designed prompt chains and validating their outputs programmatically. The engineering challenge is not in the models themselves but in the reliability, error handling, and orchestration of the multi-agent pipeline. The more precise industry term for what we built is an LLM application with agentic patterns — which is currently one of the most in-demand engineering specialisms in the market."

SECTION 10 — QUICK REFERENCE

Daily Standup (keep it under 10 minutes)

1. What did you complete yesterday?
2. What are you working on today?
3. Is anything blocked? Do you need something from me?
4. Are we on track for the week goal?

Every Friday: 30-minute demo. Show what you built that week — even if it is rough and unpolished. This keeps both of you aligned and keeps momentum going.

Definition of Done — for each task

- Code is committed to the dev branch on GitHub
- The feature works end-to-end — not just in isolation on your own machine
- No unhandled Python exceptions or browser console errors in normal flow
- The error case is handled — what if the API is down? What if the scraper fails?

Key Links to Bookmark

- Google AI Studio (free Gemini API key): aistudio.google.com
- Supabase dashboard: supabase.com/dashboard
- Gemini Python SDK docs: ai.google.dev/gemini-api/docs
- Playwright Python docs: playwright.dev/python
- FastAPI docs: fastapi.tiangolo.com
- Celery docs: docs.celeryq.dev
- Tenacity retry library: tenacity.readthedocs.io
- Google Cloud Run: cloud.google.com/run
- @hello-pangea/dnd (kanban): github.com/hello-pangea/dnd

Victoria + Heulwen — build it. Ship it.

Matchmatic v1.0